

Version

Library for handling version information and constraints

Build Status

Installation

You can add this library as a local, per-project dependency to your project using Composer:

```
composer require phar-io/version
```

If you only need this library during development, for instance to run your project's test suite, then you should add it as a development-time dependency:

```
composer require --dev phar-io/version
```

Version constraints

A Version constraint describes a range of versions or a discrete version number. The format of version numbers follows the schema of semantic versioning: `<major>.<minor>.<patch>`. A constraint might contain an operator that describes the range.

Beside the typical mathematical operators like `<=`, `>=`, there are two special operators:

Caret operator: `^1.0` can be written as `>=1.0.0 <2.0.0` and read as »every Version within major version 1«.

Tilde operator: `~1.0.0` can be written as `>=1.0.0 <1.1.0` and read as »every version within minor version 1.1. The behavior of tilde operator depends on whether a patch level version is provided or not. If no patch level is provided, tilde operator behaves like the caret operator: `~1.0` is identical to `^1.0`.

Usage examples

Parsing version constraints and check discrete versions for compliance:

```
use PharIo\Version\Version;
use PharIo\Version\VersionConstraintParser;

$parser = new VersionConstraintParser();
$caret_constraint = $parser->parse( '^7.0' );

$caret_constraint->complies( new Version( '7.0.17' ) ); // true
$caret_constraint->complies( new Version( '7.1.0' ) ); // true
$caret_constraint->complies( new Version( '6.4.34' ) ); // false
```

```
$tilde_constraint = $parser->parse( '~1.1.0' );  
  
$tilde_constraint->complies( new Version( '1.1.4' ) ); // true  
$tilde_constraint->complies( new Version( '1.2.0' ) ); // false
```

As of version 2.0.0, pre-release labels are supported and taken into account when comparing versions:

```
$leftVersion = new PharIo\Version\Version('3.0.0-alpha.1');  
$rightVersion = new PharIo\Version\Version('3.0.0-alpha.2');  
  
$leftVersion->isGreaterThan($rightVersion); // false  
$rightVersion->isGreaterThan($leftVersion); // true
```